

Algoritmo Genético de Claves Aleatorias Sesgadas (BRKGA)

1. Idea general

El código implementa un Biased Random-Key Genetic Algorithm (BRKGA) donde asignamos cada pedido a exactamente una salida válida, minimizando el tiempo máximo acumulado por zona (makespan). Las restricciones establecidas en el pdf del proyecto se verifican mediante la función `es_factible` integrada en el propio script, para así estar mas seguros de que la solución final no viola alguna regla.

2. Representación genética

Cada individuo se codifica como un vector continuo $\mathbf{r} \in [0, 1]^{|P|}$; el índice corresponde al pedido y el valor es una clave aleatoria. Ordenar los pedidos de menor a mayor clave genera la secuencia en que el decodificador intentará asignarlos. La ventaja de esta representación es que cualquier vector de claves resulta sintácticamente válido (no aparecen duplicaciones ni omisiones) de forma que el operador de cruce puede combinar genes sin provocar problemas.

3. Decodificador (función decode)

El núcleo heurístico se observa en el bucle

```
for p in orden:
```

```
    ...
```

```
    for k in salidas:
```

```
        ...
```

```
        coste = max(nueva, max(carga.values()))
```

Para cada pedido, el algoritmo evalúa todas las salidas y selecciona la que menos incrementa el makespan provisional. Al finalizar el recorrido:

- cada pedido está asignado a una salida,
- todas las cargas zonales se actualizan coherentemente,
- el makespan se obtiene como `max(carga.values())`.

Así podemos obtener factibilidad sin fases de reparación.

4. Verificación de factibilidad

La función

```
def es_factible(asigna: dict) -> bool:
```

comprueba tres condiciones indispensables:

1. correspondencia entre pedidos y salidas (un pedido a una salida),
2. pertenencia de la salida al catálogo oficial,
3. existencia de un mapeo salida→zona.

El llamado `assert es_factible(asigna)` detiene la ejecución si alguna infracción aparece.

5. Ciclo evolutivo (función `brkga`)

5.1 Población inicial

Se generan `tam_pob = 100` vectores de claves con `np.random.rand`. Cada individuo se evalúa inmediatamente mediante `decode`.

5.2 Selección elitista

Las líneas

```
pares = sorted(zip([d[2] for d in datos], poblacion),
               key=lambda x: x[0])
elite = [ind.copy() for (_, ind) in pares[:n_elite]]
```

ordenan la población por `makespan`. El 20% superior se denomina élite y se copia intacto a la siguiente generación, garantizando que la mejor calidad nunca decrezca.

5.3 Cruce sesgado (*biased crossover*)

El bloque

```
mask = np.random.rand(n_ped) < rho
hijo = np.where(mask, padre_e, padre_n)
```

implementa el cruce sesgado. Para cada gen j , la máscara booleana toma el valor `True` con probabilidad $\rho = 0,7$; cuando es verdadera, el gen se copia del padre élite, de lo contrario del padre no-élite. Así, la descendencia preserva mayor proporción de la información del padre mejor posicionado, pero mantiene variabilidad suficiente para explorar soluciones nuevas.

5.4 Mutantes

Para evitar una convergencia rápida se introducen mutantes:

```
nueva_pob.extend(np.random.rand(n_mut, n_ped))
```

El 10 % de la población se reemplaza por individuos aleatorios en cada generación, reinyectando diversidad barata.

5.5 Criterio de parada

La variable `sin_mejora` se incrementa cada vez que el mejor `makespan` no mejora en una generación; alcanzar 100 iteraciones consecutivas sin progreso detona la salida temprana, es por esto el mensaje “Convergencia temprana en la generación 318” que se ve al ejecutar el código.

6. Interpretación del parámetro ρ

El valor ρ —probabilidad de heredar el gen del padre elite—actúa como acelerador de explotación:

- $\rho \approx 0,5$: cruces equilibrados, mayor exploración, convergencia más lenta.
- $\rho \approx 0,7$ (valor elegido): énfasis moderado en el elitismo, buen balance entre calidad y diversidad.
- $\rho \rightarrow 1$: la población tiende a clonar al elite y se arriesga a estancarse prematuramente.

Resultados comparativos

El SCRIPT `comparacion_todos_algoritmos.py` ejecutó cada algoritmo varias veces sobre la misma instancia y registró, por corrida, el `makespan`¹ y el tiempo de cómputo empleado. El resumen de esas mediciones se muestra en la tabla 1; la figura 1 muestra la dispersión de los resultados y la figura 2 coloca en un mismo plano la calidad media frente al tiempo medio consumido.

Ver imagen: `boxplot_makespan.png`

Figura 1: Variabilidad del *makespan* (diagrama de cajas).

Ver imagen: `scatter_tiempo_calidad.png`

Figura 2: Relación tiempo medio–calidad media de cada algoritmo.

¹Tiempo total en la zona más cargada, entre menor sea mejor.

Cuadro 1: Indicadores estadísticos en múltiples corridas.						
Alg.	Corridas	Mejor	Prom.	Desv. std.	Gap % [†]	Tiempo (s)
GD	5	663.92	663.9	0.00	69.0	0.01
GA	20	713.05	771.5	36.4	96.4	0.01
SA	20	462.02	478.4	9.27	21.8	1.58
TS	20	392.88	393.1	0.21	0.06	18.3
BRKGA	20	392.89	392.9	0.00	0.00	73.7

Cuadro 2: *

[†] Diferencia porcentual respecto al mejor valor observado (392.88 s).

Lectura de las figuras

¿Qué muestra el box-plot? El rectángulo central representa el rango intercuartílico (IQR), es decir, la distancia entre el 25 % y el 75 % de los valores ordenados.

- Un IQR pequeño indica que la mayoría de las corridas obtienen resultados muy parecidos (alta estabilidad).
- Las bigotes prolongan ese rango hasta los valores más extremos que siguen una pauta normal: los puntos aislados serían posibles atípicos.

En la figura 1, las cajas de BRKGA y TS se reducen a una línea delgadísima: las 20 corridas entregaron exactamente el mismo *makespan*. GA, en cambio, muestra una caja alta y varios valores alejados, reflejando gran variabilidad.

¿Qué es la prueba de Kruskal–Wallis? Cuando se mide un mismo indicador en varios grupos (en nuestro caso, algoritmos) conviene comprobar si las diferencias observadas podrían ser fruto del azar. La prueba no paramétrica de Kruskal–Wallis responde justo a eso:

- Ordena todos los valores de *makespan* y asigna un rango (posición en la lista ordenada) a cada uno.
- Calcula si la distribución media de rangos difiere entre grupos.

Como no asume que los datos sigan una distribución concreta, la podemos usar aunque desconozcamos la forma en sí de los datos. En la ejecución se obtuvo $H = 77.29$ y un valor $p < 0.001$: tan pequeño que la probabilidad de que todas las diferencias sean del azar es menor del 0.1 %. Entonces, al menos un algoritmo es realmente mejor que otro con casi total seguridad.

Discusión de los resultados

1. **Calidad.** BRKGA y TS comparten el mejor *makespan* (392.9 s), muy por delante de SA (478 s), GA (771 s) y el constructivo GD (664 s).

2. **Estabilidad.** La desviación estándar de BRKGA es literalmente 0.00 s: todas las corridas llevaron al mismo óptimo. TS también es muy estable (0.21 s); GA, en contraste, puede oscilar más de 130 s entre corridas.
3. **Tiempo de cómputo.** GD y GA son casi instantáneos, pero sacrifican demasiada calidad. SA logra un balance razonable (1.6s). TS es cuatro veces más rápido que BRKGA porque manipula una única solución, mientras que BRKGA actualiza y evalúa cien soluciones por generación durante unos 300 ciclos, de ahí sus 74s promedio.

Conclusiones operativas

- Si el criterio principal es calidad garantizada, BRKGA cumple, manteniendo además la naturaleza poblacional si se interesa esto.
- Cuando el tiempo de respuesta es crítico y se acepta manipular una sola solución, TS ofrece la misma calidad en 18s (sigue siendo la mejor alternativa individual).
- Frente a métodos previos, BRKGA proporciona un salto de 22 % sobre SA, 49 % sobre GD y 50 % sobre GA.

En síntesis, los datos confirman que la implementación BRKGA es competitiva y extremadamente estable.